Відокремлений структурний підрозділ
«Економіко-технологічний фаховий коледж
Херсонського національного технічного університету»

ПРАКТИЧНА РОБОТА № 5

з дисципліни «Проєктний практикум»

Тема: «Реалізація MVP та командний Git-процес»

Проєкт «EasyStock»

Виконав:
студент групи ЗІПЗс-24
Бондаренко Сергій

Перевірив:
Дмитро Олегович Чорний
Ілля Володимирович Костевич

Хмельницький 2026

Мета та приймання наскрізного сценарію

Я підготував і перевірив наскрізний сценарій EasyStock: авторизований користувач знаходить товар, проводить надходження, отримує відмову для надмірного списання, виконує допустиме списання та перевіряє історію. Результат підтверджується API, даними SQLite і збереженням після перезапуску.

Виконав: Бондаренко Сергій. Група: ЗІПЗс-24. Формат індивідуальний: я поєдную аналіз, розроблення, тестування, інтеграцію та документацію. Комірник, менеджер і адміністратор є ролями демозастосунку, а не учасниками розробки.

Основний MVP був реалізований раніше. Новий інкремент — ізольований HTTP smoke-скрипт, перевірка збереження після рестарту, регресійний тест відповіді 503 та простежувані Git-артефакти. Перед зміною пройдено 39 API-тестів; після додавання нового тесту — 40.

Критерій	Умова приймання
Вхід і доступ	Комірник має сеанс; без сеансу 401, без ролі або CSRF — 403.
Пошук	За SSD-480 знайдено потрібну картку, початковий stock = 17 у свіжій демобазі.
Надходження	+20 повертає 201, stock = 37, документ і аудит збережено разом.
Негативний випадок	Списання 40 повертає 409; stock = 37 та лічильники записів не змінені.
Допустиме списання	−3 повертає 201, stock = 34; історія містить автора та UTC-час.
Збереження	Після повного рестарту сервера stock = 34, історія збережена.

Репозиторій EasyStock

Реалізація вертикального зрізу

Рівень	Реалізація	Доказ
React UI	Форми входу, пошуку, надходження й списання; success, error та збереження введених полів.	frontend/src/main.jsx; tests/ui.mjs
Flask API	Сеанс, серверні ролі, CSRF, формат і межі кількості.	auth.py; routes.py; validation.py
Stock Service	Атомарне збільшення; conditional decrement; один commit для залишку й аудиту.	stock_service.py
SQLite	9 сутностей, FK, UNIQUE та CHECK; журнал із залишком після операції.	models.py; database/schema.sql

API критичного сценарію

Метод і маршрут	Призначення
POST /api/auth/login	Отримати серверний сеанс і CSRF.
GET /api/products?q=SSD-480	Знайти товар та прочитати залишок.
POST /api/stock/receipt	Прийняти product_id, quantity = 20 і note.
POST /api/stock/writeoff	Відхилити 40 або провести 3 з перевіркою залишку.
GET /api/stock/history	Показати автора, час, кількість і balance_after.
POST /api/auth/logout	Відкликати серверний сеанс.

Критичні правила повторно перевіряються сервером. Помилка не змінює залишок, документ і аудит частково. Ціна в каталозі та закупівельна ціна позиції надходження зберігаються окремо.

Гілка commits та pull request

Базова main перед інкрементом: 8d2082fc432d. Для issue №1 створено окрему гілку test/1-mvp-smoke від цієї версії. Усі зміни виконано через обліковий запис chaek438.

Крок	Результат
Issue №1	Один результат, межі, критерії приймання та один виконавець.
Окрема гілка	test/1-mvp-smoke, без переписування main.
Commit smoke	Самостійний HTTP сервер, тимчасова база, критичний сценарій, рестарт.
Commit API-тесту	OperationalError → 503; rollback та приховування внутрішніх деталей.
Commit документації	Команди перевірки, фактичні журнали та два зауваження самоперевірки.
PR та інтеграція	Пов’язано з issue, наведено перевірки та відомі обмеження.

Issue №1

Гілка test/1-mvp-smoke

Коміт smoke-скрипта

Коміт тесту 503

Коміт документації

Pull request №2

Коміт інтеграції

Письмова самоперевірка належить одному автору. Вона не позначається незалежним review. Чужі commits, другий учасник і схвалення іншої людини не вигадувалися.

Позитивні та негативні перевірки

Сценарій	Очікування	Фактичний результат
Health, HTML, bundle	Сервер і готовий React доступні	200; PASS
Вхід комірника	Сеанс та CSRF	200; PASS
Без сеансу або хибний пароль	Доступ заборонено	401; PASS
Пошук SSD-480	Одна картка, stock 17	17; PASS
Запис без CSRF	БД та аудит не змінені	403; PASS
Кількість 0	БД та аудит не змінені	400; PASS
Надходження 20	Документ, stock і аудит разом	201; stock 37; PASS
Списання 40	Відмова без часткового запису	409; stock 37; PASS
Списання 3	Залишок та аудит узгоджені	201; stock 34; PASS
Історія і повторні GET	Автор, UTC, без дублів	PASS
Повний рестарт	Залишок та історія збережені	stock 34; PASS
Менеджер: операції та users	Читання дозволено, запис і users — ні	403; PASS
Вихід	Сеанс відкликано	Наступний запит 401; PASS
Штучний збій сховища	Зрозуміла помилка й rollback	503; stock 17; PASS у pytest
React DOM + HTTP API	Форми, поля після помилки, ролі	PASS

У smoke перевіряються відповіді API і прямий знімок лічильників SQLite. В окремому pytest-тесті помилку OperationalError навмисно викликано після зміни stock перед записом аудиту; перевірено відкат усієї операції.

Повторні GET підтверджують безпечне перечитування. Повторний POST після втрати відповіді не має idempotency key; перед повторною складською операцією потрібно звірити історію.

Самоперевірка та опрацювання зауважень

Виконавець самоперевірки: Бондаренко Сергій. Перевіряв відповідність issue, цілісність, помилки, права, відтворюваність і зміст diff. Нижче наведено два фактичні зауваження до повноти доказів попередньої версії.

Зауваження 1 Збереження після рестарту

Спостереження: раніше виконані API-тести доводили роботу запитів, але не завершували й повторно не запускали реальний HTTP процес. Наслідок: успішна відповідь у межах одного процесу сама по собі не підтверджувала persistence.

Опрацювання: додано smoke_mvp.py з власним Waitress, тимчасовою SQLite базою і повним restart. Після повторного входу перевірено stock = 34, кількість записів та історію. Тимчасова база очищається у finally. Результат: PASS.

Зауваження 2 Недоступність сховища

Спостереження: попередній тест відкату інжектував IntegrityError та перевіряв 409. Гілка OperationalError → 503 і приховування внутрішніх деталей не мали окремого регресійного доказу.

Опрацювання: додано test_database_failure_returns_503_and_rolls_back. Він перевіряє 503, зрозуміле повідомлення, відсутність внутрішнього шляху й типу винятку у відповіді та незмінні stock, Receipt, ReceiptItem і Movement. Результат: PASS; загальна кількість API-тестів зросла з 39 до 40.

Статус незалежного review

Розробку виконано індивідуально. Письмову самоперевірку та висновок автора збережено у PR і docs/Lab5/Review.md. Незалежний review іншою людиною не підтверджено; його потрібно фіксувати фактичним висновком рецензента.

Результати запуску та перевірка після інтеграції

Перевірка	Результат	Артефакт
Baseline API	39 passed in 18.92s	pytest-baseline.txt
API після зміни	40 passed in 19.76s	pytest.txt
Production build	PASS, Vite; 2.36 с	build.txt
HTTP + React DOM до інтеграції	PASS; 18 перевірок	smoke-before-merge.json
HTTP після інтеграції	PASS; 17 перевірок	smoke-after-merge.json
Ізоляція та очищення	Тимчасову базу видалено	temporary_database_removed = true

Дата початкового прогону: 15.10.2026. Середовище: Linux, Python 3.12, Node.js, SQLite, Flask/SQLAlchemy та Waitress. Докази збережено як текст і JSON без сеансових токенів.

Після злиття виконано новий HTTP прогін із новою тимчасовою базою на ізольованій копії перевірених файлів. Це повторна перевірка інтегрованого коду; вона не є твердженням про запуск на іншому фізичному ПК.

Фрагмент журналу

```
PASS: receipt 20 -> balance 37
PASS: writeoff 40 -> HTTP 409, balance unchanged
PASS: writeoff 3 -> balance 34
PASS: author and UTC timestamp in history
PASS: full process restart -> balance 34
PASS: manager writeoff -> HTTP 403
PASS: logout -> HTTP 401
PASS: temporary database removed
```

Це стислий виклад фактичних результатів; оригінальні журнали збережені у docs/Lab5/evidence. Показники часу стосуються цього локального середовища і не є вимірюванням часу роботи людини.

Відтворюваний запуск і конфігурація

Для запуску готового демо потрібен Python 3.11 або новіший. Node.js 22.12+ потрібен для перескладання frontend і React DOM-тестів. Спочатку потрібно повністю отримати та розпакувати проєкт.

Windows

```
run_demo.bat
```

Після встановлення залежностей відкрити http://127.0.0.1:5000. Ручний запуск із кореня проєкту:

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
.venv\Scripts\python.exe start.py --demo
```

Відтворення smoke test

```
.venv\Scripts\python.exe scripts\smoke_mvp.py
```

Скрипт самостійно запускає і зупиняє сервер, використовує тимчасову базу та не приймає адресу робочого складу. Для React DOM попередньо виконати npm ci в frontend, потім додати --with-ui.

Linux або macOS

```
bash run_demo.sh
.venv/bin/python scripts/smoke_mvp.py --with-ui
```

Змінні середовища

.env.example — довідковий шаблон. start.py автоматично не читає .env; DATABASE_URL і COOKIE_SECURE потрібно задавати як змінні процесу. Для окремої БД потрібен абсолютний SQLite шлях. Зміна наявної схеми потребує міграції; create_all її не перебудовує.

У цій перевірці використано ізольовану копію коду з уже встановленими залежностями. Повторне отримання пакетів з Інтернету та інший фізичний комп’ютер окремо не перевірялися.

Демонстрація та межі перевірки

План демонстрації розраховано на 4 хвилини 30 секунд. Показ виконується на окремій свіжій демобазі, де SSD-480 має 17 одиниць. За вже змінених даних потрібно користуватися фактичним залишком.

Час	Дія	Що показати
0:00–0:30	Увійти як комірник	Огляд складу та дозволене меню
0:30–1:00	Знайти SSD-480	Картка й початковий залишок 17
1:00–1:45	Прийняти 20	Повідомлення успіху і залишок 37
1:45–2:30	Спробувати списати 40	Помилка; поля та залишок 37 збережені
2:30–3:15	Списати 3 і відкрити історію	Залишок 34, автор, час і дві операції
3:15–4:00	Увійти як менеджер	Читання дозволено; складські дії недоступні
4:00–4:30	Відкрити GitHub і журнали	Issue, гілка, PR, commits та результати

Відомі обмеження

1. React DOM перевіряє введення, події та API, але не CSS, мобільну верстку або сумісність Chromium/Firefox.

2. Незалежний людський walkthrough, вимірювання часу пошуку та review іншою людиною залишаються непідтвердженими.

3. Після втрати відповіді на POST немає idempotency key; перед повтором треба звірити історію.

4. SQLite підходить для локального MVP; кілька процесів і високе навантаження потребують окремого рішення.

Власні фактичні знімки інтерфейсу готую під час локального демопрогону. Текстові результати підтверджують перевірки, перелічені у журналах.

Висновки та контрольні запитання

Я перевірив наявний вертикальний сценарій EasyStock та додав відтворювану HTTP перевірку з перезапуском процесу. Позитивні й негативні випадки підтверджують цілісність залишку та аудиту. Окремий тест 503 захищає зрозумілу реакцію на недоступність сховища без внутрішніх деталей у відповіді.

Git-артефакти пов’язують критерії, тематичну гілку, commits, PR і журнали. Розробку та самоперевірку виконав один автор. Незалежні людські перевірки мають отримати окремі фактичні результати.

Запитання	Відповідь
Чому commits поділено саме так?	Smoke-скрипт, регресійний тест 503 і документація мають різні перевірювані результати, тому збережені окремо.
Яке зауваження покращило результат?	Перевірка повного рестарту доповнила перевірку поточного процесу; тест 503 підтвердив rollback і зрозумілу помилку без внутрішніх деталей.
Як обробляються помилки та доступ?	400 — некоректна кількість; 401 — немає сеансу; 403 — немає ролі/CSRF; 409 — недостатній залишок; 503 — недоступна БД.
Як відтворити запуск без автора?	Отримати код, створити venv, встановити backend/requirements.txt і запустити start.py --demo; готовий frontend/dist уже включено.
Чим відрізняється smoke після merge?	Він повторно перевіряє інтегрований стан і перезапуск сервера; попередній прогін підтверджує лише версію до інтеграції.
Що не зберігати у Git і як діяти за витоку?	Не додавати приватні ключі, .env, cookie та робочі БД. Реальний секрет відкликати; очищення історії виконувати за погодженою процедурою.

Використані матеріали

Матеріали викладача «Реалізація MVP та командний Git процес»: методичний текст на 15 сторінках і презентація на 55 слайдах.

Репозиторій проєкту: https://github.com/chaek438/EasyStock

Задача перевірки MVP №1

Pull request №2